Introduction to Artificial Intelligence

Jinshuo Li

Shanghai Jiao Tong University

January 18, 2026

Contents

Preface

Standing at the beginning of 2026, looking back at the tumultuous development of Artificial Intelligence over the past few years, one cannot help but feel a sense of awe. From the initial explosion of generative models to the ubiquity of intelligent agents today, AI has fundamentally reshaped the way we research, work, and live.

However, amidst the dazzling array of applications and the rapid iteration of tools, the underlying principles often remain obscured by the hype. As a student at Shanghai Jiao Tong University, I have spent much of my time navigating through this fog, attempting to connect the classical theories of machine learning with the emergent behaviors of modern large models.

This document is not merely a collection of technical notes; it is an attempt to construct a coherent narrative of AI knowledge as I understand it. It begins with the fundamental mathematics and algorithms, traverses through the architectures that defined the deep learning era, and arrives at the frontier of current research.

My hope is that this work will serve as a roadmap for those who, like me, are eager to look beyond the "black box" and understand the elegant logic that drives the intelligence of our time.

Writing is a process of learning. All errors and omissions are my own, and I welcome any corrections.

Jinshuo Li
Shanghai Jiao Tong University
2026.1.6, in Beijing

Chapter 1

Basics

This is the first chapter of the document. Here we will cover the basic concepts and foundational information necessary for understanding the subsequent chapters.

1.1 Language and Packages

To follow the trends in research of artificial intelligence, we will use **Python** as the programming language throughout this book. Python is widely adopted in the AI community due to its simplicity and the vast array of libraries available for machine learning, data analysis, and scientific computing.

To be specific, I would like to inform the readers that we use **3.11.9** version of Python in this book at the time of January 18, 2026. You can download it from the official Python website: `https://www.python.org/downloads/release/python-3119/`. This version is stable and includes the latest features and improvements, while maintaining compatibility with most libraries used in AI research.

Also, we will use several popular Python libraries throughout the book, including but not limited to:

- **NumPy**: For numerical computations and array manipulations.
- **Pandas**: For data manipulation and analysis.
- **Matplotlib** and **Seaborn**: For data visualization.
- **Pytorch** and **TensorFlow**: For building and training machine learning models.

We have to say that this should be a continuously updating list as new libraries and tools are developed in the fast-evolving field of AI. I encourage readers to stay updated with the latest developments and consider exploring additional libraries that may be relevant to their specific interests and projects.

1.2 Introduction to Tensor

A **tensor** is a mathematical object that generalizes the concepts of scalars, vectors, and matrices to higher dimensions. Tensors are fundamental in representing data in various forms, especially in deep learning frameworks like TensorFlow and PyTorch. You can simply think of tensors as multi-dimensional arrays. It is similar to **ndarray** in **NumPy**, which is a powerful library for numerical computing in Python. Just like ndarrays, tensors can hold data of various types (e.g., integers, floats) and can be manipulated using a variety of operations.

To follow the trends, we use **PyTorch** tensors in this book. PyTorch is a popular deep learning framework that provides efficient tensor operations and automatic differentiation, making it easier to build and train neural networks. The most remarkable feature of PyTorch tensors is their ability to leverage GPU acceleration for faster computations, which is crucial for training large-scale machine learning models.

Python Code: Basic tensors usage in PyTorch

```python
import torch

# Arange tensor
tensor_x = torch.arange(0, 10, step=2)
tensor_y = torch.zeros((2,3,4))
tensor_w = torch.rand((2,2))

# Creating a tensor
tensor_a = torch.tensor([[1, 2], [3, 4]])

# Properties of the tensor
print("Shape:", tensor_a.shape)
print("Size:", tensor_a.numel())
print("Total:", tensor_a.sum().item())
# tensor.item() to get the value as a standard Python number

# Tensor operations (Different to Linear Algebra)
tensor_b = tensor_a.reshape(4, 1)
tensor_c_group = (tensor_a + tensor_b, tensor_a - tensor_b, tensor_a * tensor_b,
    tensor_a / tensor_b, tensor_a ** tensor_b)
tensor_d = (tensor_b == tensor_a)  # Element-wise operations, returns a new
    tensor stuffed with boolean values.

# Tensor Concatentation
tensor_d = torch.cat((tensor_a, tensor_a), dim=0)  # Concatenate after rows
tensor_e = torch.cat((tensor_a, tensor_a), dim=1)  # Concatenate after columns

# Tensor Broadcasting
tensor_f = tensor_a + torch.tensor([1, 2])

# Tensor Indexing and Slicing
element = tensor_a[0, 1]  # Accessing a single element
row = tensor_a[1, :]      # Accessing a row
column = tensor_a[:, 0]   # Accessing a column
sub_tensor = tensor_a[0:2, 0:2]  # Slicing

# Tensor Type Conversion
tensor_h = tensor_a.int()    # Convert to int type, float type is the same.
ndarray_from_tensor = tensor_a.numpy()  # Convert to NumPy ndarray
```

Remark 1.2.1

Here are a few additional notes for the code above on tensors in PyTorch:

- Broadcasting addition as an example, adds $[1, 2]$ to each row of `tensor_a`. It's a process of expanding (duplicating) the smaller tensor to match the shape of the larger tensor during arithmetic operations.
- PyTorch tensors support a wide range of operations, including mathematical functions, linear algebra operations, and more.
- Tensors can be moved between CPU and GPU memory for efficient computation using the '.to(device)' method.
- PyTorch provides automatic differentiation capabilities through the 'autograd' module, which is essential for training neural networks.

We will introduce more details about tensors and their applications in deep learning as we progress through the book. Understanding tensors is crucial for effectively working with machine learning models and implementing various algorithms.

We encourage readers to experiment with tensors in PyTorch and explore their capabilities further.

More information about PyTorch tensors can be found in the official documentation: `https://pytorch.org/docs/stable/tensors.html`.

1.3 Data Preparation

Data preparation is a crucial step in the machine learning pipeline, as the quality and structure of the data can significantly impact the performance of the models. In this section, we will discuss some common techniques for preparing data for machine learning tasks.

1.3.1 Data Reading

For example, if we want to read a CSV file containing our dataset, we can use the Pandas library in Python:

Firstly, we need to create a sample CSV file named `house_tiny.csv` in the `data` directory.

Python Code: Read files

```python
import os

os.makedirs(os.path.join('..', 'data'), exist_ok=True)
data_file = os.path.join('..', 'data', 'house_tiny.csv')
with open(data_file, 'w') as f:
    f.write('NumRooms,Alley,Price\n')  # Header for each column
    f.write('NA,Pave,127500\n')        # Each row represents a data sample
    f.write('2,NA,106000\n')
    f.write('4,NA,178100\n')
    f.write('NA,NA,140000\n')
```

Now, we can read the CSV file using Pandas:

Python Code: Read CSV file using Pandas

```python
import pandas as pd

data = pd.read_csv(data_file)
print(data)
```

The output will be:

	NumRooms	Alley	Price
0	NA	Pave	127500
1	2	NA	106000
2	4	NA	178100
3	NA	NA	140000

1.3.2 Handling Missing Values

In real-world datasets, it is common to encounter missing values. There are several strategies to handle missing data, including:

- **Removing rows or columns**: If a row or column has a significant amount of missing data, it may be best to remove it entirely.
- **Imputation**: Filling in missing values with statistical measures such as the mean, median, or mode of the respective column.
- **Using algorithms that support missing values**: Some machine learning algorithms can handle missing data without any preprocessing.

Here we consider to use mean imputation for numerical columns and mode imputation for categorical columns.

Python Code: Handling Missing Values

```
inputs, outputs = data.iloc[:, 0:2], data.iloc[:, 2]
inputs = inputs.fillna(inputs.mean())
print(inputs)
```

This way we have handled the missing values in the **inputs**. The output will be:

	NumRooms	Alley
0	3.0	Pave
1	2.0	NaN
2	4.0	NaN
3	3.0	NaN

We can also view **NaN** as a category in the **Alley** column. Then we can transform the categorical variable into the form of:

Python Code: Categorical variable encoding

```
inputs = pd.get_dummies(inputs, dummy_na=True)
print(inputs)
```

The output will be:

	NumRooms	Alley_NA	Alley_Pave
0	3.0	0	1
1	2.0	1	0
2	4.0	1	0
3	3.0	1	0

1.3.3 Convert the Type to Tensor

After handling missing values and encoding categorical variables, we can convert the data into tensors for further processing in machine learning models.

Python Code: Convert DataFrame to Tensors

```
import torch

X = torch.tensor(inputs.to_numpy(), dtype=torch.float32)
Y = torch.tensor(outputs.to_numpy(), dtype=torch.float32)

print(X)
print(Y)
```

Then we have successfully converted the prepared data into PyTorch tensors, which can be used for training machine learning models.

1.4 Some Linear Algebra

In this section, we will review some fundamental concepts of linear algebra that are essential for understanding machine learning algorithms.

1.4.1 Vectors

A **vector** is an ordered collection of numbers that can be represented as a point in a multi-dimensional space. Vectors can be added together and multiplied by scalars, and they are often used to represent data points, features, or weights in machine learning. In **PyTorch**, vectors are represented as one-dimensional tensors.

Python Code: i

```
mport torch

vector_a = torch.tensor([1, 2, 3])
vector_b = torch.tensor([4, 5, 6])
```

We can visit each element of the vector using indexing:

Python Code: Vector Indexing

```
first_element = vector_a[0]
print("First element of vector_a:", first_element)
```

Just like arrays in NumPy, PyTorch tensors support various operations on vectors, such as addition, subtraction, dot product, and more.

Dot Product

The **dot product** (or scalar product) of two vectors is a mathematical operation that takes two equal-length sequences of numbers and returns a single number. The dot product is calculated by multiplying corresponding elements of the vectors and summing the results.

Python Code: Dot Product

```
dot_product = torch.dot(vector_a, vector_b)
print("Dot product of vector_a and vector_b:", dot_product)
```

1.4.2 Matrix

A **matrix** is a two-dimensional array of numbers arranged in rows and columns. Matrices are used to represent data, transformations, and relationships between variables in machine learning. In **PyTorch**, matrices are represented as two-dimensional tensors.

Python Code: i

```
mport torch

matrix_a = torch.tensor([[1, 2, 3], [4, 5, 6]])
matrix_b = torch.tensor([[7, 8, 9], [10, 11, 12]])
```

We can access elements of the matrix using row and column indices:

Python Code: Matrix Indexing

```
element_1_2 = matrix_a[1, 2]  # Accessing element in the 2nd row and 3rd column
print("Element at (1, 2) of matrix_a:", element_1_2)
```

We can perform matrix transposition, multiplication, and other operations using PyTorch functions.

Python Code: Operations on Matrices

```
matrix_c = matrix_a.T  # Transpose of matrix_a
```

1.4.3 Tensor

A **tensor** is a multi-dimensional array that generalizes the concepts of scalars (0D), vectors (1D), and matrices (2D) to higher dimensions. Tensors can have any number of dimensions and are used to represent complex data structures in machine learning and deep learning. In **PyTorch**, tensors are the primary data structure for storing and manipulating data.

Python Code: i

```
mport torch

tensor_a = torch.arrange(24).reshape(2, 3, 4)  # A 3D tensor with shape (2, 3, 4)
```

We can access elements of the tensor using multiple indices corresponding to each dimension:

Python Code: Tensor Indexing

```
element_1_2_3 = tensor_a[1, 2, 3]  # Accessing element in the 2nd block, 3rd row,
    and 4th column
print("Element at (1, 2, 3) of tensor_a:", element_1_2_3)
```

We can clone tensors to create a copy of the tensor:

Python Code: Cloning Tensors

```
tensor_b = tensor_a.clone()
```

Remark 1.4.1

The .clone() method is a deepcopy operation that creates a new tensor with its own memory allocation. This means that any modifications made to the cloned tensor will not affect the original tensor, and vice versa. This is particularly useful when you want to preserve the original data while performing operations or transformations on a copy of the tensor.

We can also perform various operations on tensors, such as addition, multiplication, and more. We have to know that the all the binary operations between two tensors in PyTorch are element-wise operations. The multiplication between two tensors is not the matrix multiplication but the element-wise multiplication. For matrix multiplication, we have to use the `torch.matmul()` function or the `@` operator.

Dimensionality Reduction

Dimensionality reduction is a technique used to reduce the number of features or dimensions in a dataset while preserving its essential characteristics. This is particularly useful in machine learning, where high-dimensional data can lead to overfitting and increased computational complexity.

One common method is summing over specific dimensions of a tensor. In PyTorch, we can use the `torch.sum()` function to achieve this.

Python Code: Dimensionality Reduction by Summation

```
import torch

tensor_a = torch.arange(24).reshape(2, 3, 4)  # A 3D tensor with shape (2, 3, 4)
reduced_tensor_a = torch.sum(tensor_a, dim=0)  # Summing over the first dimension
reduced_tensor_b = torch.sum(tensor_a, dim=1)  # Summing over the second
    dimension
reduced_tensor_c = torch.sum(tensor_a, dim=2)  # Summing over the third dimension
```

We can also use the `keepdim` parameter to retain the reduced dimensions with size one, which can be useful for broadcasting in subsequent operations.

Python Code: Dimensionality Reduction with keepdim

```
reduced_tensor_d = torch.sum(tensor_a, dim=1, keepdim=True)  # Summing over the
    second dimension and keeping the dimension
```

Norm

The **norm** of a tensor is a measure of its magnitude or length. In PyTorch, we can compute various types of norms using the `torch.norm()` function. The most commonly used norms are the L_1 norm (sum of absolute values) and the L_2 norm (Euclidean norm).

More specifically, for a tensor A, the L_1 norm is defined as:

$$\|A\|_1 = \sum_{i,j,k} |a_{i,j,k}|$$

and the L_2 norm is defined as:

$$\|A\|_2 = \sqrt{\sum_{i,j,k} a_{i,j,k}^2}$$

Here is an example of computing the L_1 and L_2 norms of a tensor in PyTorch:

Python Code: Computing Norms of Tensors

```python
import torch
tensor_a = torch.tensor([[1.0, -2.0, 3.0], [-4.0, 5.0, -6.0]])
l1_norm = torch.norm(tensor_a, p=1)  # L1 norm
l2_norm = torch.norm(tensor_a, p=2)  # L2 norm
```

1.5 Some Analysis

In this section, we will review some fundamental concepts of analysis that are essential for understanding machine learning algorithms.

About 2500 years ago, the ancient Greek mathematician Eudoxus of Cnidus developed the method of exhaustion, which laid the groundwork for integral calculus. Later, in the 17th century, Isaac Newton and Gottfried Wilhelm Leibniz independently developed the formal framework of calculus, introducing concepts such as limits, derivatives, and integrals.

Just as we can see, the most important question in analysis, is **optimization**, which is the process of finding the best solution from a set of possible solutions. In machine learning, optimization is used to minimize a loss function, which measures the difference between the predicted output and the actual output.

1.5.1 Derivation

In monadic calculus, the **derivative** of a function measures how the function changes as its input changes. It is a fundamental concept in optimization, as it provides information about the slope of the function at a given point.

$$f'(x) = \lim_{\Delta \to 0} \frac{f(x+\Delta) - f(x)}{\Delta}$$

In multivariate calculus, the **partial derivative** of a function with respect to one of its variables measures how the function changes as that variable changes, while keeping the other variables constant.

$$\frac{\partial f}{\partial x_i} = \lim_{\Delta \to 0} \frac{f(x_1, \ldots, x_i + \Delta, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{\Delta}$$

1.5.2 Gradient

The **gradient** of a multivariate function is a vector that contains all of its partial derivatives. It points in the direction of the steepest ascent of the function and is used in optimization algorithms to find the minimum or maximum of a function.

$$\nabla_x f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)^T$$

For gradient, we have following properties:

- $\forall A \in \mathbb{R}^{m \times n}, \nabla_x Ax = A^T$
- $\forall A \in \mathbb{R}^{n \times m}, \nabla_x x^T A = A$
- $\forall A \in \mathbb{R}^{n \times n}, \nabla_x x^T Ax = (A + A^T)x$
- $\nabla_x ||x||^2 = \nabla_x (x^T x) = 2x$

These properties are useful in deriving gradients for various functions commonly encountered in machine learning and optimization problems.

1.5.3 Chain Rule

The **chain rule** is a fundamental theorem in calculus that allows us to compute the derivative of a composite function. If we have two functions f and g, the chain rule states that the derivative of the composite function $h(x) = f(g(x))$ is given by:

$$h'(x) = f'(g(x)) \cdot g'(x)$$

In the context of multivariate functions, the chain rule can be extended to compute the gradient of a composite function. If $f : \mathbb{R}^m \to \mathbb{R}$ and $g : \mathbb{R}^n \to \mathbb{R}^m$, then the gradient of the composite function $h(x) = f(g(x))$ is given by:

$$\nabla_x h(x) = J_g(x)^T \nabla_y f(y)\big|_{y=g(x)}$$

where $J_g(x)$ is the Jacobian matrix of g at x.

1.5.4 Automatic Differentiation

In machine learning, we often need to compute gradients of complex functions with respect to their inputs. **Automatic differentiation** is a technique that allows us to compute these gradients efficiently and accurately. It works by breaking down the computation of a function into a series of elementary operations, each of which has a known derivative. By applying the chain rule systematically, we can compute the gradient of the entire function.

Here is a simple example of automatic differentiation using **PyTorch**:

Python Code: Automatic Differentiation in PyTorch

```python
import torch

# Create a tensor with gradient tracking enabled
x = torch.arange(4.0, requires_grad=True)

# Define a function of x
y = 2 * torch.dot(x, x)

# Compute the gradient of y with respect to x
y.backward()

# Print the computed gradient, stored in x.grad as a property of the tensor x
print("Gradient of y with respect to x:", x.grad)
```

The output will be:

```
Gradient of y with respect to x: tensor([ 0.,  4.,  8., 12.])
```

This indicates that the gradient of y with respect to x is $[0, 4, 8, 12]$, which corresponds to the derivative of the function $y = 2(x_0^2 + x_1^2 + x_2^2 + x_3^2)$ evaluated at $x = [0, 1, 2, 3]$.

If we want to calculate the result of a function with two or more variables, and we only want to calculate the gradient with respect to one of them, we can use the `.detach()` parameter in the process to retain the computation graph for further gradient calculations.

Python Code: Automatic Differentiation with Multiple Variables

```python
import torch

# Create tensors with gradient tracking enabled
x = torch.arange(4.0, requires_grad=True)
y = torch.arange(1.0, 5.0, requires_grad=True)

# Define a function of x and y
# Detach y to prevent gradient tracking for y
z = torch.dot(x, y.detach())

# Compute the gradient of z with respect to x
z.backward()

# Print the computed gradient, stored in x.grad as a property of the tensor x
print("Gradient of z with respect to x:", x.grad)
```

Then we can get the gradient of z with respect to x only.

1.5.5 Probability

Simply speaking, machine learning is all about making predictions based on data. Since data is often noisy and uncertain, we need to use probability theory to model this uncertainty and make informed predictions. Probability provides a mathematical framework for quantifying uncertainty and making decisions based on incomplete information.

We shall introduce some basic concepts of probability that are relevant to machine learning.

Definition 1.5.1

The Axioms of Probability In a given sample space S, the probability function $P(A)$ satisfies the following axioms:

- Non-negativity: For any event A, $P(A) \geq 0$.
- Normalization: The probability of the entire sample space is 1, i.e., $P(S) = 1$.
- Additivity: For any countable sequence of mutually exclusive events $A_1, A_2, A_3, \ldots$, the probability of their union is equal to the sum of their individual probabilities:

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

Definition 1.5.2

Conditional Probability The conditional probability of an event A given that another event B has occurred is defined as:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad \text{if } P(B) > 0$$

Definition 1.5.3

Bayes' Theorem Bayes' theorem relates the conditional probabilities of two events A and B:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}, \quad \text{if } P(B) > 0$$

Definition 1.5.4

Marginal Probability The marginal probability of an event A is the total probability of A occurring, regardless of the occurrence of other events. It can be computed by summing or integrating over the probabilities of all possible outcomes that include A:

$$P(A) = \sum_i P(A \cap B_i) \quad \text{or} \quad P(A) = \int P(A \cap B) dB$$

where B_i represents all possible events that can occur alongside A.

Definition 1.5.5

Expectation and Variance The expectation (or expected value) of a random variable X is a measure of its central tendency and is defined as:

$$E[X] = \sum_x x \cdot P(X = x) \quad \text{(for discrete random variables)}$$

$$E[X] = \int x \cdot f_X(x)\, dx \quad \text{(for continuous random variables)}$$

The variance of a random variable X measures the spread of its values around the expectation and is defined as:

$$Var(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$$

1.6 More Informations

Here, we provide some ways to find special properties.

Python Code: Finding Special Properties for Specific Modules

```
import torch

# For example, to find all the properties and methods of the torch.nn module
print(dir(torch.nn))
```

Python Code: Finding Documentation for Specific Variables

```
import torch

x = torch.arange(4.0, requires_grad=True)
help(torch.x)
```

Then we can get the documentation for the variable `x`.

Chapter 2

Linear Regression

Regression is a fundamental concept that can modelling the relationship between a dependent variable and one or more independent variables. Linear regression is one of the simplest and most widely used regression techniques. It assumes a linear relationship between the input features and the output variable.

2.1 Basic Concepts

2.1.1 Linear Model

Traditional linear model can be represented as:

$$y = w^T x + b$$

where y is the predicted output, x is the input feature vector, w is the weight matrix, and b is the bias term.

More rigorously speaking, the formula above is a kind of **affine transformation**, which is a linear transformation followed by a translation. In this case, the linear transformation is represented by the weight matrix w, and the translation is represented by the bias term b.

2.1.2 Loss Function

The loss function quantifies the difference between the predicted output and the actual output. In linear regression, the most commonly used loss function is defined as:

$$l^{(i)} = \frac{1}{2}(y^{(i)} - \hat{y}^{(i)})^2$$

where $l^{(i)}$ is the loss for the i-th training example, $y^{(i)}$ is the actual output, and $\hat{y}^{(i)}$ is the predicted output.

To measure the overall performance of the model on the entire training dataset, we can use the Mean Squared Error (MSE) loss function, which is defined as:

$$L = \frac{1}{m}\sum_{i=1}^{m} l^{(i)} = \frac{1}{2m}\sum_{i=1}^{m}(y^{(i)} - \hat{y}^{(i)})^2$$

where L is the overall loss, and m is the number of training examples.

The overall purpose is to find the optimal parameters w and b that minimize the loss function L.

2.1.3 Analytical Solution for Linear Regression